Parallel Voxelization of Triangle Meshes

1st Jose Francisco Paca Sotero
Faculty of Computing
University of Engineering and Technology (UTEC)
Lima, Peru
jose.paca@utec.edu.pe

2nd Diva Stewart Maquera Bobadilla
Faculty of Computing
University of Engineering and Technology (UTEC)
Lima, Peru
diva.maquera@utec.edu.pe

3rd Daniel Ignacio Casquino Paz
Faculty of Computing
University of Engineering and Technology (UTEC)
Lima, Peru
daniel.casquino@utec.edu.pe

***Abstract*—This paper presents a parallel implementation of conservative voxelization for triangle meshes. The main implementation uses MPI to distribute triangles across processes, build private bit-packed voxel grids, and merge the final result with a bitwise OR reduction. To justify the selected paradigm, two shared-memory OpenMP variants are also implemented: one with a private grid per thread and one with a shared grid updated through atomic OR operations. Finally, a minimal CUDA atomic prototype is evaluated on Khipu to measure whether directly porting the atomic strategy to GPU is worthwhile. The evaluation compares execution time, speedup, efficiency, and experimental overhead on synthetic models and on the Stanford Bunny mesh.**

***Index Terms*—voxelization, MPI, OpenMP, CUDA, parallel computing, overhead, PRAM**

I. Introduction

Voxelization is the process of discretizing a 3D object into an axis-aligned grid of volumetric pixels. This representation is used in computer graphics, medical imaging, urban analysis, simulation, and interactive applications. This project studies triangle-mesh voxelization as a parallel computing problem: given a set of triangles and a fixed grid resolution, mark the voxels occupied by the mesh.

The first version of the project modeled the algorithm in PRAM and identified task parallelism over triangles. This final version keeps that idea but implements and measures three concrete CPU variants: MPI, OpenMP with private grids, and OpenMP with atomic updates. A fourth CUDA atomic prototype is added as an experimental comparison, while optimized tile-based or sparse GPU voxelization is discussed as future work because it requires additional spatial indexing.

II. Algorithm

A. Input and Representation

The input is a triangle mesh $T = \{t_0, t_1, \ldots, t_{m-1}\}$ and a grid resolution r. The voxel grid has $V = r^3$ cells. To reduce memory, the grid is stored as a bitset implemented with 64-bit words, so the number of stored words is $w = \lceil V/64 \rceil$.

For each triangle, the current implementation computes its axis-aligned bounding box (AABB) in grid coordinates and marks every voxel inside that box. This is conservative: it can mark false positives, but it is simple, deterministic, and exposes parallel work clearly.

B. PRAM Model

The high-level algorithm can be modeled as CRCW PRAM because different workers may write to the same voxel. The write conflict is benign when the operation is a logical OR, as marking the same voxel multiple times still leaves the final bit set to one.

Algorithm 1 Voxelization (PRAM)

```
B ← empty bitset
T ← triangulate(model.load(path)).faces
for all t in T pardo do
    for all candidate voxel v inside AABB of t do
        i ← index(v)
        B[i] ← B[i] ∨ 1
    end for
end for
return B
```

c_i is defined as the number of candidate voxels visited for triangle t_i. Then, $C = \sum_{i=0}^{m-1} c_i$. The sequential work of the voxelization stage is:

$$T_s = \Theta(m + C)$$

With unbounded PRAM processors over triangles, the span is dominated by the largest triangle AABB:

$$T_\infty = O(\max_i c_i)$$

Using Brent's theorem, an ideal parallel bound is then:

$$T_p = O\left(\frac{m + C}{p} + \max_i c_i\right)$$

This bound explains the main limitation of triangle partitioning: if a few triangles cover very large AABBs, load balancing can degrade.

TABLE I
FOSTER DESIGN SUMMARY.

Stage	Decision
Partitioning	Split the triangle list into ranges
Communication	Broadcast/scatter triangles; OR-reduce grids
Agglomeration	Pack voxels into 64-bit words
Mapping	One range per rank/thread

TABLE II
MODELS USED IN THE EVALUATION.

Model	Triangles	Role
Triangle/tetra	1–4	Smoke tests
Blocky/skeleton	72–96	Visual and CUDA portability
Stanford Bunny	69,451	Scaling benchmark

C. Foster Design

The implementation follows Foster's four design stages. In the partitioning stage, the independent work units are triangles; each unit generates the candidate voxels inside one AABB. Communication consists of distributing triangles and combining the local bitsets with OR. Agglomeration groups many triangles per worker and packs 64 voxels per word to reduce memory and communication. Mapping assigns contiguous triangle ranges to MPI ranks or OpenMP threads. This is coarse-grained task parallelism over triangles, not a tile-based spatial decomposition.

III. PARALLEL IMPLEMENTATIONS

A. MPI

The MPI implementation is the main distributed-memory version. Rank zero loads the mesh and distributes triangles with `MPI_Scatterv`. Then, each rank builds its own local bitset grid. The final grid is obtained with `MPI_Reduce` and `MPI_BOR`.

$$T_p^{\text{MPI}} \approx \frac{C}{p} + T_{\text{scatter}}(m,p) + T_{\text{reduce}}(w,p)$$

This design avoids data races during voxel marking, as each process writes to private memory only. Its main overhead comes from process initialization, triangle distribution, synchronization, and reduction of the bit-packed grid.

B. OpenMP with Private Grids

The `omp_private` variant uses one process and multiple threads. Each thread writes to a private grid, and all grids are merged with an OR reduction:

$$T_p^{\text{OMP-private}} \approx \frac{C}{p} + T_{\text{reduce\_mem}}(w,p)$$

This version avoids atomic operation overhead, but it increases memory usage by a factor that is close to the number of threads.

C. OpenMP with Atomic Updates

The `omp_atomic` variant uses a shared bitset grid. Each update uses an atomic OR on the 64-bit word that contains the target voxel bit.

Here we must also define a term that represents the overhead of atomic operations. U is defined as the number of grid updates across all threads, while T_{atomic} is the additional time that is required for a single atomic operation. Additionaly, the contention term models the additional delay that might occur when multiple threads access the same word (cache coherence).

$$T_p^{\text{OMP-atomic}} \approx \frac{C}{p} + UT_{\text{atomic}} + T_{\text{contention}}$$

This version saves memory, but several threads can contend for the same 64-bit word even when they update different voxel bits.

D. CUDA with Atomic Updates

The `cuda_atomic` prototype ports the shared-grid atomic strategy to the GPU. It copies the triangle array to device memory, allocates a bit-packed grid on the GPU, launches one CUDA thread per triangle, and uses `atomicOr` for each candidate voxel bit. The result is copied back to the CPU and checked with the same occupancy metrics as the CPU modes.

This version is intentionally minimal. It answers whether a direct GPU port of the current AABB algorithm is enough. It does not implement tile binning, block-private grids, prefix sums, or sparse structures.

IV. EXPERIMENTAL METHODOLOGY

The measured region is `voxel_seconds`. For each mode, the baseline T_s is the same mode with $p = 1$. The metrics are:

$$S(p) = \frac{T_s}{T_p}, \qquad E(p) = \frac{S(p)}{p}, \tag{1}$$

$$T_o(p) = pT_p - T_s, \qquad R_o(p) = \frac{T_o(p)}{T_s}. \tag{2}$$

The experiments use several models with different roles. Small OBJ models are used for correctness and cross-platform tests because they load without Assimp on Khipu. The Stanford Bunny is used as the larger benchmark because it has $69{,}451$ triangles and exposes the effect of computation versus overhead.

Local multi-mode experiments used `blocky_creeper_like`, grid resolutions 32, 64, and 128, $p \in \{1, 2, 4\}$, and three repetitions per configuration. MPI scaling was also measured on the Stanford Bunny for $p \in \{1, 2, 4, 8\}$.

V. RESULTS

For resolutions 32 and 64, the measured times are very small and overhead dominates. At resolution 128, MPI obtains its best local point at $p = 2$. OpenMP atomic improves up to $p = 4$ in this small mesh, while OpenMP private does not clearly outperform the other variants because the private-grid reduction is significant relative to the computation.

Fig. 1. Z-projection images of voxelized Bunny, creeper-like, and skeleton-like models.

TABLE III
LOCAL RESULTS FOR RESOLUTION 128, MEDIAN OF THREE RUNS.

Mode	p	T_p (s)	$S(p)$	$E(p)$	O/T_s
MPI	1	0.002703	1.000	1.000	0.000
MPI	2	0.001755	1.540	0.770	0.299
MPI	4	0.002288	1.181	0.295	2.386
OMP atomic	1	0.003745	1.000	1.000	0.000
OMP atomic	2	0.002837	1.320	0.660	0.515
OMP atomic	4	0.002061	1.817	0.454	1.201
OMP private	1	0.003016	1.000	1.000	0.000
OMP private	2	0.002748	1.098	0.549	0.822
OMP private	4	0.002948	1.023	0.256	2.910
OMP atomic CUDA (local)	1	0.304685	—	—	—

The Bunny results show the same strong-scaling pattern from the course: for a fixed problem, the best p is finite. At $r = 128$, the larger triangle count amortizes MPI overhead up to $p = 4$. At $r = 512$, the dense grid reduction and memory traffic dominate after $p = 2$ in the measured laptop run.

VI. KHIPU VALIDATION

The project was also validated on UTEC's Khipu cluster using Slurm, OpenMPI, and CMake. The code was adjusted so it can build without Assimp by using a small OBJ fallback loader for benchmark models. A conservative debug job used up to four CPUs and small resolutions. This run is treated as portability validation, not as a definitive benchmark, because the measured times are in the microsecond to millisecond range and are therefore sensitive to scheduler and environment noise.

One operational issue was observed: the available OpenMPI environment did not work correctly with direct `srun` execution for this program. The stable approach was to reserve resources with Slurm and use `mpirun` inside the allocation.

The CUDA prototype produced the same occupied voxel counts as the CPU versions, but it was slower for the tested small and medium cases. This supports the decision not to use this direct CUDA port as the main delivery.

VII. DISCUSSION

The results show that increasing p does not always reduce execution time. For strong scaling, the problem size is fixed, so local computation decreases approximately as C/p, but reduction, synchronization, launch, and memory costs do not disappear. Therefore, the best p is not necessarily the largest available p.

TABLE IV
MPI RESULTS ON THE STANFORD BUNNY.

r	p	T_p (s)	$S(p)$	$E(p)$
128	1	0.012897	1.000	1.000
128	2	0.007408	1.741	0.871
128	4	0.004490	2.873	0.718
128	8	0.005638	2.288	0.286
512	1	0.078667	1.000	1.000
512	2	0.062191	1.265	0.632
512	4	0.085296	0.922	0.231
512	8	0.105886	0.743	0.093

TABLE V
KHIPU GPU DEBUG RESULTS FOR THE BLOCKY MODEL.

Mode	r	T_p (s)	Tested voxels
MPI $p = 1$	128	0.000639	277944
OMP atomic $t = 4$	128	0.002261	277944
CUDA atomic	128	0.311759	277944
MPI $p = 1$	512	0.022754	4380856
OMP atomic $t = 4$	512	0.042784	4380856
CUDA atomic	512	0.410128	4380856

MPI is defensible as the main implementation because it represents distributed memory directly, avoids concurrent writes through private grids, and expresses the final merge as a natural bitwise OR reduction. OpenMP is simpler on one machine and can have lower overhead, but its variants expose different costs: private grids spend memory and reduction bandwidth, while atomic updates can suffer contention.

The main weakness of triangle-based partitioning is load balance. If a mesh has only a few large triangles, some workers may receive much more work than others. A tile-based algorithm could assign disjoint voxel regions to workers and eliminate the final OR reduction, but a naive tile implementation would compare too many triangles against too many voxels. Its work is $\Theta(mr^3/p)$ per parallel step without spatial filtering, while the current triangle-based implementation visits only $C = \sum_i c_i$ candidate voxels. For example, a mesh with $69{,}451$ triangles at $r = 512$ would imply about 9.3×10^{12} triangle-voxel tests in the naive all-triangles-per-tile approach. Practical tile-based voxelizers need spatial filtering structures such as binning, BVH, octrees, or sparse grids.

A. Why CUDA Was Not the Main Delivery

CUDA is technically attractive, especially when a local GPU is available, and the implemented `cuda_atomic` prototype confirms that the project can run on Khipu's Tesla T4 GPU. However, the limiting issue is not only whether a kernel can be launched. A fast GPU version needs a better memory layout, less atomic contention, transfer timing, and output validation against the CPU implementation.

The memory cost also grows quickly for dense grids. A bit-packed grid requires $r^3/8$ bytes, while a byte-per-voxel grid requires r^3 bytes. At $r = 4096$, the bit-packed grid already needs 8 GiB, and a byte grid needs 64 GiB. If each GPU block or worker stores four private bit-packed grids before reducing,

the same $r = 4096$ case needs 32 GiB, above a 12 GiB local GPU. With 12 GiB of VRAM, the theoretical maximum dense bit-packed grid is about $r = 4688$ for a single grid, about $r = 2953$ with four private grids, and about $r = 2344$ with eight private grids, before counting triangles, auxiliary buffers, CUDA runtime overhead, and temporary reductions.

These limits were tested locally on an NVIDIA GeForce RTX 5070, which has 12 GiB of VRAM. The `cuda_atomic` prototype ran successfully for cases up to $r = 2048$, but it failed for $r = 4096$ and above due to memory limitations.

Therefore, CUDA is reasonable future work only if paired with a better memory strategy, such as tile binning, block-level reductions, or sparse grids. For this delivery, MPI and OpenMP remain the main defensible comparison of distributed and shared memory overheads, while `cuda_atomic` is reported as a validated but non-competitive prototype.

VIII. Conclusion

The final implementation keeps MPI as the main solution, adds OpenMP variants for comparison, and includes a minimal CUDA atomic prototype. The experiments confirm that overhead is central to the analysis: for small grids, parallelism can lose to overhead, while for larger fixed grids some configurations obtain speedup. The CUDA prototype validates GPU execution but shows that a direct atomic port is not enough for these inputs. Future work should replace the AABB marking approximation with a triangle-box test and implement tile-based GPU partitioning with spatial filtering.

Use of AI

OpenAI ChatGPT/Codex was used as an assistant to organize the report, review the complexity derivation, compare MPI/OpenMP/CUDA tradeoffs, and prepare documentation. The code, formulas, and conclusions were checked against the repository implementation, official MPI/OpenMP/Khipu documentation, and the technical references below. AI was not used as a primary source for experimental results.

References

[1] M. Schwarz and H.-P. Seidel, "Fast parallel surface and solid voxelization on GPUs," *ACM Transactions on Graphics*, vol. 29, no. 6, Article 179, 2010.

[2] T. Akenine-Moller, "Fast 3D triangle-box overlap testing," *Journal of Graphics Tools*, vol. 6, no. 1, pp. 29–33, 2001.

[3] I. Foster, *Designing and Building Parallel Programs: Concepts and Tools for Parallel Software Engineering*. Addison-Wesley, 1995.

[4] Open MPI Project, "MPI_Scatterv(3), MPI_Bcast(3), and MPI_Reduce(3) manual pages," Open MPI Documentation. [Online]. Available: https://www.open-mpi.org/doc/

[5] OpenMP Architecture Review Board, "OpenMP API Specification," 2024. [Online]. Available: https://www.openmp.org/specifications/

[6] Open Asset Import Library, "Assimp: Open Asset Import Library." [Online]. Available: https://www.assimp.org/

[7] UTEC Khipu, "Khipu documentation." [Online]. Available: https://docs.khipu.utec.edu.pe/

[8] OpenAI, "ChatGPT/Codex," large language model, 2026. [Online]. Available: https://chat.openai.com/